

Física Computacional (106018C)

Taller [N.2º] — Estructuras de Control, Ciclos y Condicionales C³

Semana [N.2º] — Lenguaje: [Python]

[Jordy Castillo Jordán] — [24/09/2026]

1. Contexto físico. Se estudió el decaimiento radiactivo de $N_0 = 10\,000$ núcleos, con una vida media $t_{1/2} = 5$ años, siendo un proceso exponencial útil para aplicaciones como la datación con carbono-14. Se calcula $N(t)$ durante 30 años y el tiempo en que permanece el 50 %, 25 % y 10 % de la muestra.

2. Método. Se evaluó la exponencial de decaimiento con NumPy sobre un arreglo de 150 tiempos (de 0 a 30 años). Para los tiempos asociados a una fracción dada se despejó t aplicando logaritmo. Así mismo, se calculó la constante λ .

$$N(t) = N_0 e^{-\lambda t}, \quad \lambda = \frac{\ln 2}{t_{1/2}} \approx 0,14 \text{ años}^{-1}, \quad t = -\frac{\ln(N/N_0)}{\lambda}.$$

Como N/N_0 es la fracción de la muestra que permanece, un único ciclo `for` sobre las fracciones 0,50, 0,25 y 0,10 evalúa los tres tiempos con la misma expresión.

3. Resultado. A continuación se comparan los valores obtenidos por el código para los valores de vida media y tiempos de los porcentajes pedidos, teniendo en cuenta el valor de referencia.

Caso	Núcleos restantes $N(t)$			Caso	Tiempo t (años)		
	Obtenido	Referencia	ER(%)		Obtenido	Referencia	ER(%)
1 vida media (5 años)	5000	5000	0,00	50 % de N_0	5,00	5,00	0,00
2 vidas medias (10 años)	2500	2500	0,00	25 % de N_0	10,00	10,00	0,00
3 vidas medias (15 años)	1250	1250	0,00	10 % de N_0	16,61	16,61	0,00

Verificación (PASS/FAIL): 6 de 6 casos de la celda de verificación pasaron (3 de la Parte A y 3 de la Parte B). La igualdad es total porque $N(t)$ se evalúa directamente con su forma cerrada, por lo cual no hay error de truncamiento. Los resultados presentan un error relativo de 0,00 %, confirmando una precisión correcta en los algoritmos planteados.

4. Obstáculo o decisión de implementación. Al sumar la serie de Taylor con $x = 100$ sin reducir el argumento, el resultado no pasaba la verificación, pese a tener el criterio inicial correcto, ya que la cancelación destruía los dígitos significativos en el proceso. En lugar de aumentar `max_iter`, se aplicó $\sin(x + 2\pi m) = \sin(x)$ con `math.remainder(x, 2*math.pi)`, y $x = 100$ y $x = -237$ pasaron la verificación. Además, comparar la serie recursiva con la ingenua exigió usar por primera vez `try/except` (ante un posible `OverflowError`, que no ocurrió en los valores probados), lo que generó errores y confusión hasta que la celda funcionó bien. Para hallar los tiempos del 50 %, 25 % y 10 %, no bastó con despejar t ; hubo que deducir que el cociente N/N_0 dentro del logaritmo es justamente el porcentaje en forma decimal, lo que permitió resolver los tres casos con un solo ciclo `for`.

5. Conclusión. Se repasaron conceptos de `matplotlib` y se descubrieron otros para mejorar la presentación, como ubicar puntos sobre una curva para marcar las vidas medias. En un futuro se podrían fusionar pasos (la figura del punto 5 repite el código de la del punto 4) para no expandir el código. Además, se aprendió que dos códigos procesos equivalentes pueden compartirse diferente en el programa, como refleja la Parte A. En general, lo desarrollado se podrá incorporar de la mejor forma en métodos más elaborados en un futuro.

6. Uso de herramientas de IA. Se usó Gemini (NotebookLM) para averiguar cómo ubicar puntos sobre una gráfica, cómo escribir una ecuación tipo cociente en una celda de texto del notebook y la corrección del código para crear la tabla en el reporte. Se usó Claude para revisar el código frente a la teoría de las presentaciones y en búsqueda de contradicciones, siendo lo más relevante la verificación PASS/FAIL de la Parte A con reducción de argumento.

Código fuente: ver archivo adjunto `FC_Taller02_JordyCastillo.ipynb`.